



TRANSFORMACIÓN DE TEXTO NATURAL A EMBEDDINGS

73.64 - TEMAS AVANZADOS EN DEEP LEARNING - 2025

Ing. Eugenia Piñeiro, Ing. Marina Fuster

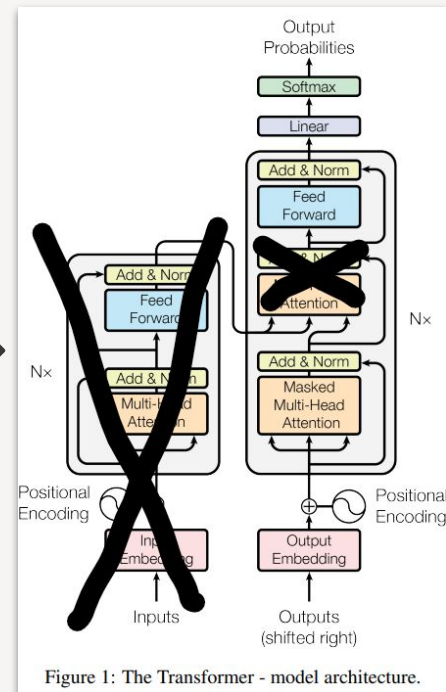
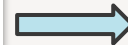
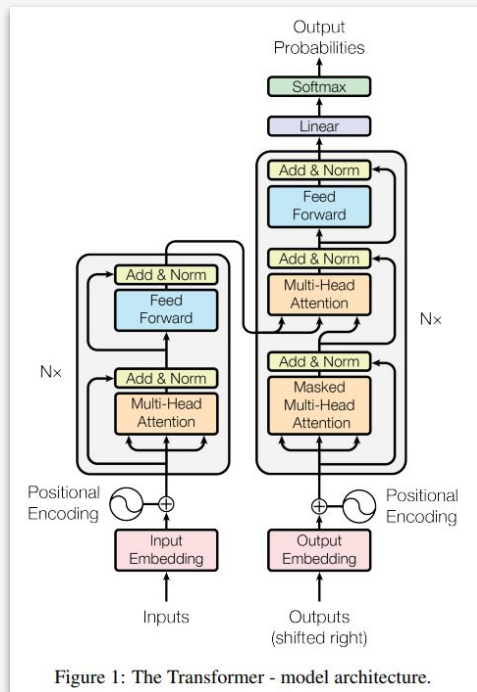
01

DEMO

¿Cómo se implementa un transformer y el mecanismo de atención?

DEMO

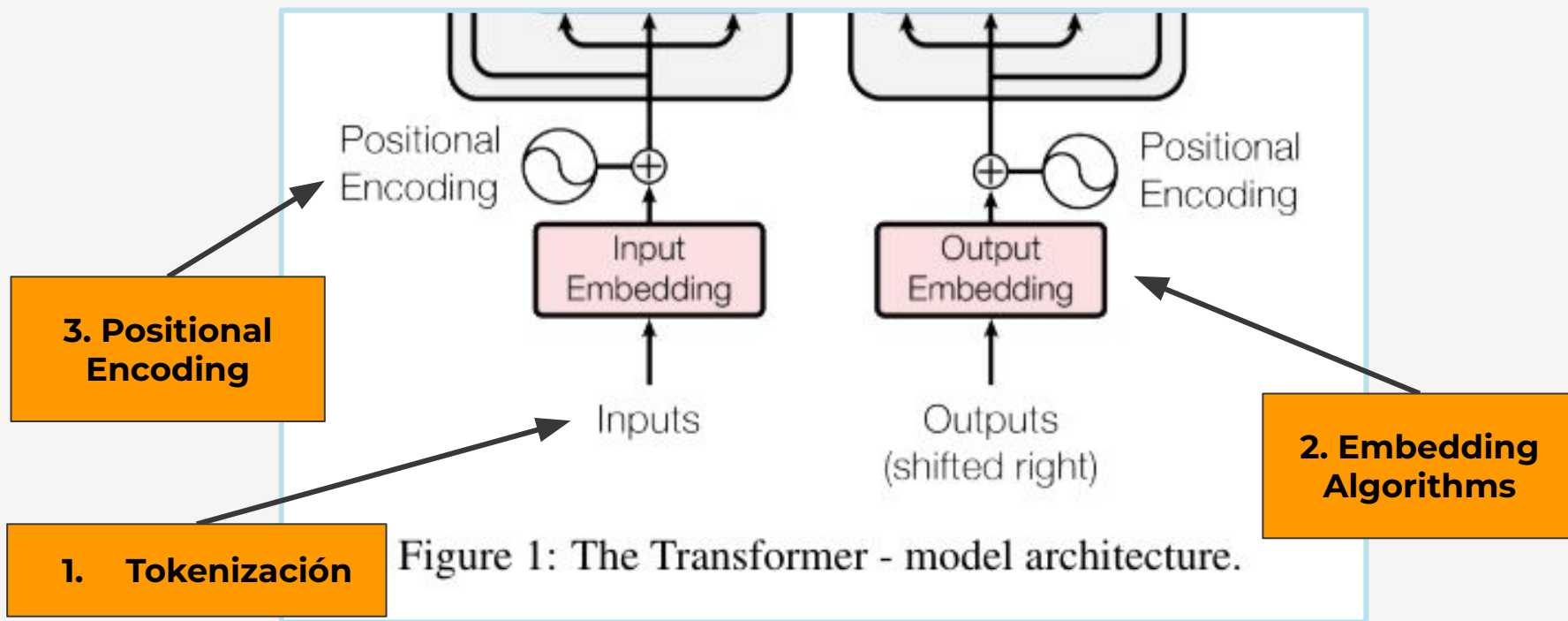
Demo - Decoder Transformer



02

TRANSFORMACIÓN DEL TEXTO NATURAL

LAS ETAPAS DEL DATO DE ENTRADA



03

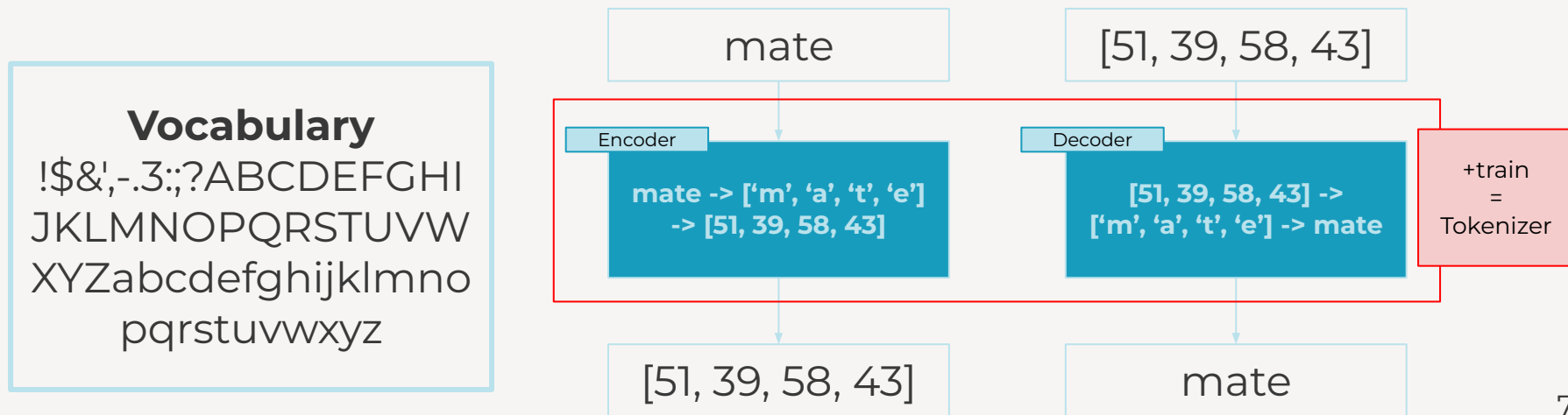
TOKENIZERS

¿Qué unidades de procesamiento definimos para el transformer?

¿Qué es la tokenización?

División de los elementos de un texto (frase, documento, etc) de acuerdo a un **vocabulario** predefinido.

Mediante la **tokenización** se generan las unidades que consume un transformer (importante para los costos).



Vocabulary
.../gaseosa/mate/
agua/té/jugo/...

mate

Encoder

[156]

[156]

Decoder

mate

Vocabulary
.../pe/me/te/
pa/ma/ta/...

mate

Encoder

[41, 39]

[41, 39]

Decoder

mate

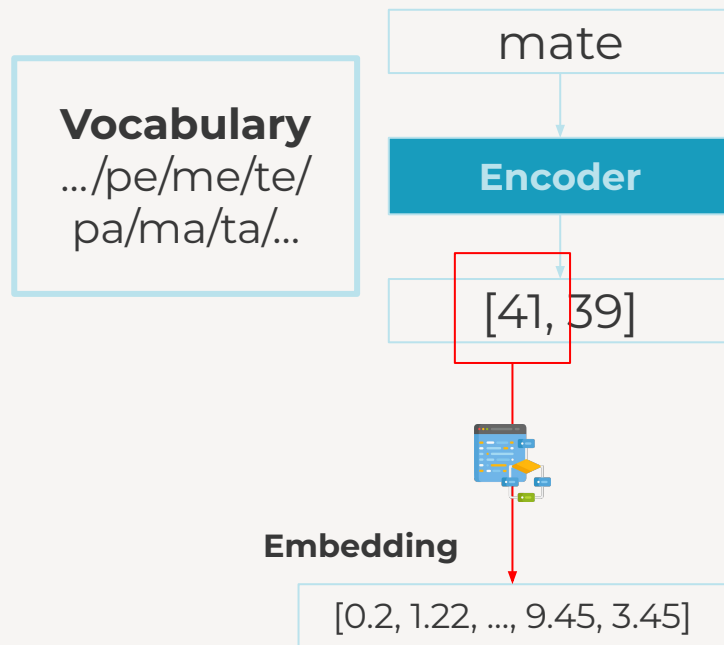
Spoiler: ¿Cuál es la diferencia con embedding?

Tokenizer

Parte el texto en unidades más pequeñas para que pueda ser procesado por un modelo.

Algoritmo de Embedding

Crea vectores densos que representan unidades de texto (con el objetivo de capturar representaciones de interés)



TIPOS DE TOKENS

- **Básicos:** tokens comunes que representan palabras, sub-palabras, puntuación, espacios, etc.
- **Especiales (reservados):** aquellos que tienen un significado especial o función asignada

Tiktokenizer

System ▾ You are a helpful assistant ✕

User ▾ I want to go to Brazil tomorrow ✕

Assistant ▾ Sure, here's... ✕

Add message

Token count
84

```
<|im_start|>system<|im_sep|>You are a helpful assistant<|im_end|><|im_start|>user<|im_sep|>I want to go to Brazil tomorrow<|im_end|><|im_start|>assistant<|im_sep|>Sure, here's...<|im_end|><|im_start|>assistant<|im_sep|>
```

```
<|im_start|>system<|im_sep|>You are a helpful assistant<|im_end|>
<|im_start|>user<|im_sep|>I want to go to Brazil tomorrow<|im_end|>
<|im_start|>assistant<|im_sep|>Sure, here's...<|im_end|>
<|im_start|>assistant<|im_sep|>
```

BPE (algoritmo de tokenización)

Github Repository: <https://github.com/openai/tiktoken>

Live Tokenizer: <https://tiktokenizer.vercel.app/?model=gpt2>

The screenshot shows the GitHub repository page for 'tiktoken' by user 'hauntsaninja'. The repository is public and has 10.9k stars, 737 forks, and 162 watchers. It is currently on the 'main' branch with 2 other branches and 11 tags. The repository description states: 'tiktoken is a fast BPE tokenizer for use with OpenAI's models.' The file list includes: .github/workflows (Bump cibuildwheel, last month), scripts (Bump version, sync codebase, last year), src (Simplify byte_pair_merge (#255), 4 months ago), tests (Sync codebase, last year), tiktoken (Sync codebase, last month), tiktoken_ext (Sync codebase, last month), .gitignore (Sync codebase, last year), and CHANGELOG.md (Sync codebase, last year). The commit history shows a recent update to the README to mention gpt-4o by c0ba74c last month, with 47 commits in total. The right sidebar shows the 'About' section with links to Readme, MIT license, Activity, and Custom properties, and a 'Releases' section.

tiktoken Public

Watch 162 Fork 737 Star 10.9k

main 2 Branches 11 Tags

Go to file Add file Code

hauntsaninja update README to mention gpt-4o c0ba74c · last month 47 Commits

File	Commit	Time
.github/workflows	Bump cibuildwheel	last month
scripts	Bump version, sync codebase	last year
src	Simplify byte_pair_merge (#255)	4 months ago
tests	Sync codebase	last year
tiktoken	Sync codebase	last month
tiktoken_ext	Sync codebase	last month
.gitignore	Sync codebase	last year
CHANGELOG.md	Sync codebase	last year

About

tiktoken is a fast BPE tokenizer for use with OpenAI's models.

- Readme
- MIT license
- Activity
- Custom properties

10.9k stars
162 watching
737 forks

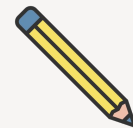
Releases

BPE (algoritmo de tokenización): “*train*”



Hola, hoy tengo mucha hambre 🍔🍔🍔🍔🍔🍔

BPE (algoritmo de tokenización): “*train*”



Hola, hoy tengo mucha hambre 🍔🍔🍔🍔🍔🍔



\u0048\u006f\u006c\u0061\u002c\u0020\u0068\u006f\u0079\u0020\u0074\u0065\u006e\u0067\u006f\u0020\u006d\u0063\u0068\u0061\u006d\u0062\u0072\u0065\u0020\u2615\u2615\u2615\u2615\u2615\u2615



72-111-108-97-44-32-104-111-121-32-116-101-110-103-111-32-109-117-99-104-97-32-104-97-109-98-114-101-32-240-159-141-148-240-159-141-148-240-159-141-148-240-159-141-148-240-159-141-148-240-159-141-148



BPE (algoritmo de tokenización): “**train**”

Hola, hoy tengo mucha hambre 🍔🍔🍔🍔🍔🍔



\u0048\u006f\u006c\u0061\u002c\u0020\u0068\u006f\u0079\u0020\u0074\u006
5\u006e\u0067\u006f\u0020\u006d\u0075\u0063\u0068\u0061\u0020\u0068\u006
6\u006d\u0062\u0072\u0065\u0020\u00d8\u00c\u00df54\u00d8\u00c\u00df54\u00d8\u00c\u00df54\u00d8
3\u00c\u00df54\u00d8\u00c\u00df54\u00d8\u00c\u00df54



72-111-108-97-44-32-104-111-121-32-116-101-110-103-111-32-109-117-99-104-97-32-104-97-1
09-98-114-101-32-240-159-141-148-240-159-141-148-240-159-141-148-240-159-141-148-2
40-159-141-148-240-159-141-148



BPE (algoritmo de tokenización): “*train*”

Hola, hoy tengo mucha hambre 🍔🍔🍔🍔🍔🍔



\u0048\u006f\u006c\u0067\u002c\u0020\u0068\u006f\u0079\u0020\u0074\u0065\u006e\u0067\u006f\u0020\u006d\u0075\u0063\u0068\u0067\u0020\u0068\u0067\u006d\u0062\u0072\u0065\u0020\u2d83c\u2df54\u2d83c\u2df54\u2d83c\u2df54\u2d83c\u2df54\u2d83c\u2df54



72-111-108-97-44-32-104-111-121-32-116-101-110-103-111-32-109-117-99-104-97-32-104-97-1
09-98-114-101-32-240-256-148-240-256-148-240-256-148-240-256-148-240-256-148-2
40-256-148

BPE (algoritmo de tokenización):

Language Models are Unsupervised Multitask Learners

lapping training data with test evaluation tasks.

2.2. Input Representation

A general language model (LM) should be able to compute the probability of (and also generate) any string. Current large scale LMs include pre-processing steps such as lower-casing, tokenization, and out-of-vocabulary tokens which restrict the space of model-able strings. While processing Unicode strings as a sequence of UTF-8 bytes elegantly fills this requirement as exemplified in work such as Gilmer et al. (2015), current byte-level LMs are not competitive with word-level LMs on large scale datasets such as One Billion Word Benchmark (AI-Rfou et al., 2018). We observed a similar performance gap in our own attempts to train standard byte-level LMs on WebText.

Byte Pair Encoding (BPE) (Sennrich et al., 2015) is a practical middle ground between character and word level language modeling which effectively interpolates between word level inputs for frequent symbol sequences and character level inputs for infrequent symbol sequences. Despite its name, reference BPE implementations often operate on Unicode code points and not byte sequences. These implementations would require including the full space of Unicode symbols in order to model all Unicode strings. This would result in a base vocabulary of over 130,000 before any multi-symbol tokens are added. This is prohibitively large compared to the 32,000 to 64,000 token vocabularies often used with BPE. In contrast, a byte-level version of BPE only requires a base vocabulary of size 256. However, directly applying BPE to the byte sequence results in sub-optimal merges due to BPE using a greedy frequency heuristic for building the token vocabulary. We observed BPE including many versions of common words like `dog` since they occur in many variations such as `dog .` `dog!` `dog?`. This results in a sub-optimal allocation of limited vocabulary slots and model capacity. To avoid this, we prevent BPE from merging across character categories for any byte sequence. We add an exception for spaces which significantly improves the compression efficiency while adding only minimal fragmentation of words across multiple vocabularies.

Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

Table 2. Architecture hyperparameters for the 4 model sizes.

BPE only requires a base vocabulary of size 256. However, directly applying BPE to the byte sequence results in sub-optimal merges due to BPE using a greedy frequency based heuristic for building the token vocabulary. We observed BPE including many versions of common words like `dog` since they occur in many variations such as `dog .` `dog!` `dog?`. This results in a sub-optimal allocation of limited vocabulary slots and model capacity. To avoid this, we prevent BPE from merging across character categories for any byte sequence. We add an exception for spaces which significantly improves the compression efficiency while adding only minimal fragmentation of words across multiple vocabularies.

The model was manually tuned for the best perplexity on a 5% held-out sample of WebText. All models still underfit WebText and held-out perplexity has as of yet improved given more training time.

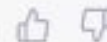
Problemas de tokenización: glitch tokens



What does the string " SolidGoldMagikarp" refer to?



The word "distribute" refers to the act of distributing or spreading something out among a group of people or things. It can also refer to the way in which something is distributed or arranged. For example, a company may distribute its products to different retailers, or a teacher may distribute assignments to students. Additionally, it can mean to give out something in an orderly way, such as distributing flyers or pamphlets.



[SolidGoldMagikarp \(plus, prompt generation\)](#)

Trade-offs de agrandar el vocabulario

- Permite obtener secuencias más cortas, útil para la capa de atención.
- Términos poco frecuentes pueden modelarse como tokens individuales
- Permiten modelar mejor otros lenguajes (si acompaña el dataset)



¿Cómo **usuario**, qué me conviene?

Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies

Chaofan Tao^{1,2} Qian Liu^{2†} Longxu Dou^{2†} Niklas Muennighoff^{3,4}
Zhongwei Wan⁵ Ping Luo¹ Min Lin² Ngai Wong^{1†}

¹The University of Hong Kong ²Sea AI Lab ³Contextual AI
⁴Stanford University ⁵The Ohio State University

Abstract

Research on scaling large language models (LLMs) has primarily focused on model parameters and training data size, overlooking the role of vocabulary size. We investigate how vocabulary size impacts LLM scaling laws by training models ranging from 33M to 3B parameters on up to 500B characters with various vocabulary configurations. We propose three complementary approaches for predicting the compute-optimal vocabulary size: IsoFLOPs analysis, derivative estimation, and parametric fit of the loss function. Our approaches converge on the conclusion that the optimal vocabulary size depends on the compute budget, with

Byte Latent Transformer: Patches Scale Better Than Tokens

Artidoro Pagnoni, Ram Pasunuru[†], Pedro Rodriguez[‡], John Nguyen[‡], Benjamin Muller, Margaret Li^{1,◊}, Chunting Zhou[◊], Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman^{†,2,◊}, Srinivasan Iyer[†]

FAIR at Meta, ¹Paul G. Allen School of Computer Science & Engineering, University of Washington,

²University of Chicago

[‡]Joint second author, [†]Joint last author, [◊]Work done at Meta

We introduce the Byte Latent Transformer (BLT), a new byte-level LLM architecture that, for the first time, matches tokenization-based LLM performance at scale with significant improvements in inference efficiency and robustness. BLT encodes bytes into dynamically sized patches, which serve as the primary units of computation. Patches are segmented based on the entropy of the next byte, allocating more compute and model capacity where increased data complexity demands it. We present the first FLOP controlled scaling study of byte-level models up to 8B parameters and 4T training bytes. Our results demonstrate the feasibility of scaling models trained on raw bytes without a fixed vocabulary. Both training and inference efficiency improve due to dynamically selecting long patches when data is predictable, along with qualitative improvements on reasoning and long tail generalization. Overall, for fixed inference costs, BLT shows significantly better scaling than tokenization-based models, by simultaneously growing both patch and model size.

Date: December 16, 2024

Correspondence: artidoro at [cs.washington.edu](mailto:artidoro@cs.washington.edu), sviyer at [meta.com](mailto:sviyer@meta.com)

Code: <https://github.com/facebookresearch/blt>

04

EMBEDDING ALGORITHMS

¿Cómo representamos el
significado de un elemento
lingüístico?

¿Cómo representamos el significado de una palabra?

Es de interés modelar:

- ¿Qué palabras son sinónimos?
- ¿Cuáles antónimos?
- ¿Cuáles tienen connotación positiva?
- ¿Y negativa?
- ¿Cuáles son palabras que pertenecen al mismo concepto base?
- ...

¿Cómo representamos el significado de una palabra?

Es de interés modelar:

- ¿Qué palabras son sinónimos?
- ¿Cuáles antónimos?
- ¿Cuáles tienen connotación positiva?
- ¿Y negativa?
- ¿Cuáles son palabras que pertenecen al mismo concepto base?
- ...

Lexical
Semantics

Vector Semantics

- Surge de la convergencia de dos ideas:
 - Joos (1950), Harris (1954) y Firth (1957), definen el significado por su **distribución**: “*You shall know a word by the company it keeps*”
 - Osgood (1957), quien usa un **vector en $\mathbf{R}^3$** para representar connotaciones.

	Valence	Arousal	Dominance
courageous	8.05	5.5	7.38
music	7.67	5.57	6.5
heartbreak	2.45	5.65	3.58
cub	6.71	3.95	4.24

[Jurafsky, D., & Martin, J. H. \(2023\). *Speech and Language Processing* \(3rd ed. draft\).](#)

Primeras Implementaciones

- Matriz término-documento

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.3 The term-document matrix for four words in four Shakespeare plays. The red boxes show that each document is represented as a column vector of length four.

Primeras Implementaciones

- Matriz término-término

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 6.6 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.

[Jurafsky, D., & Martin, J. H. \(2023\). *Speech and Language Processing* \(3rd ed. draft\).](#)

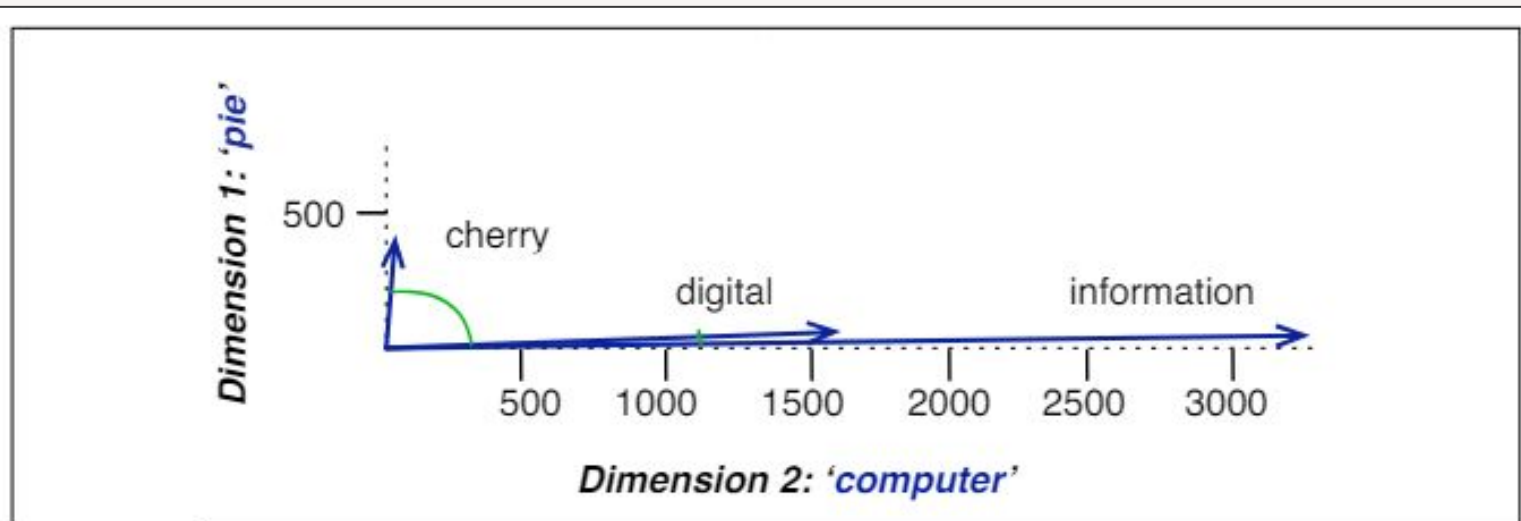
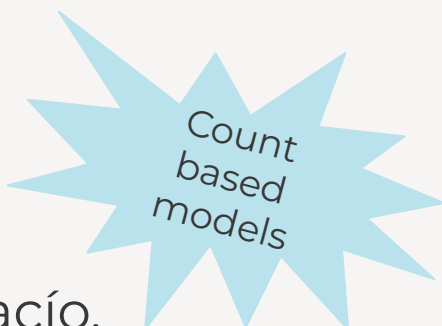


Figure 6.8 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

¿Qué problemas tienen estas implementaciones?



Count
based
models

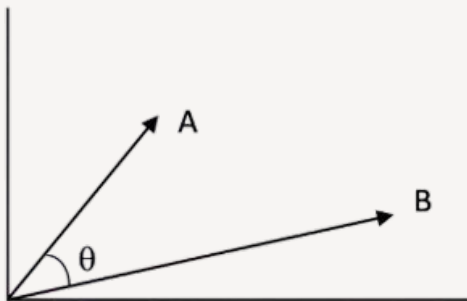
- Los vectores dispersos tienen mucho espacio vacío.
- La frecuencia sola no alcanza: ¿qué ocurre con palabras como “a”, “de”, “y”?
- ¿Y con palabras que se usan muy poco?

Embedding

- Vector n -dimensional.
- Representan elementos discretos (palabras, caracteres, imágenes) en un espacio continuo.
- Buena parte de la literatura los define como densos.

¿Cómo medir similitud entre embeddings?

Cosine Similarity



$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

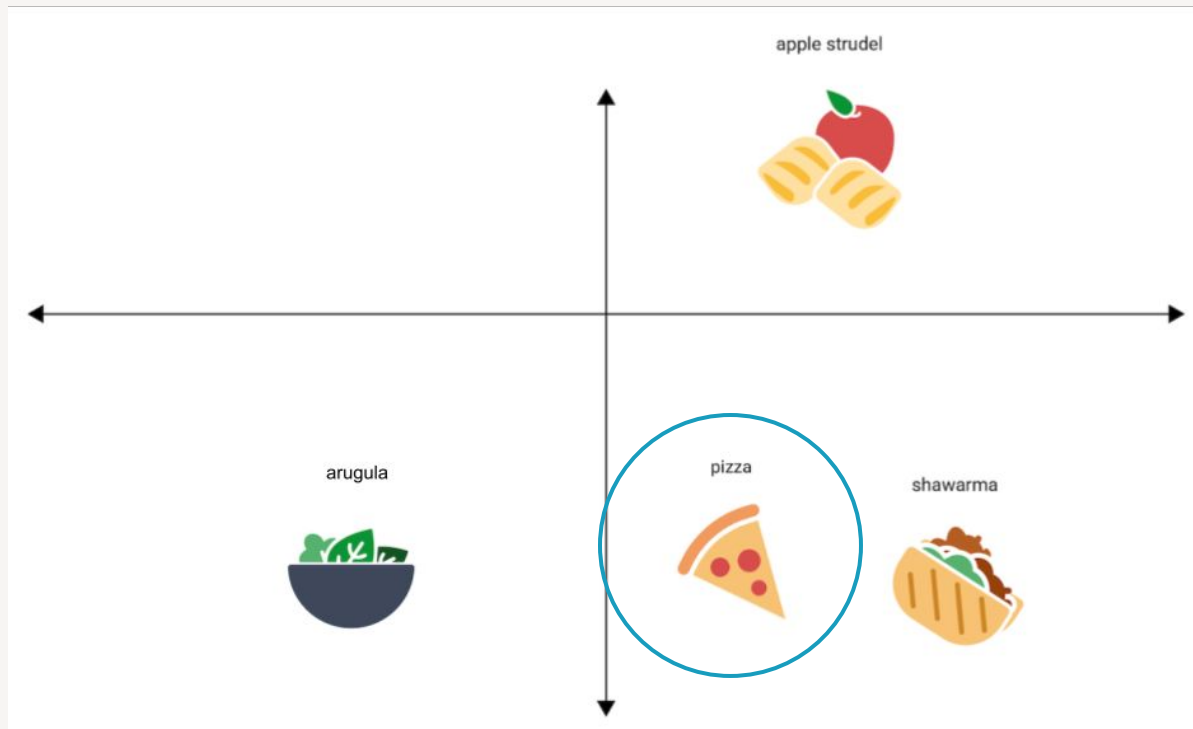
Tipos de embedding (vectores densos)

- **Static Embedding:** mantienen un valor fijo que se aprendió del contexto en el corpus de datos.
 - word2vec (skip-gram y CBOW)
- **Dynamic Embeddings:** cambia su valor dependiendo del contexto en una instancia particular.
 - BERT (palabras), SBERT (oraciones)
- **Learned Embeddings:** se entrenan junto con el modelo (como en la demo!)

STATIC EMBEDDING

“I ate **pizza**”

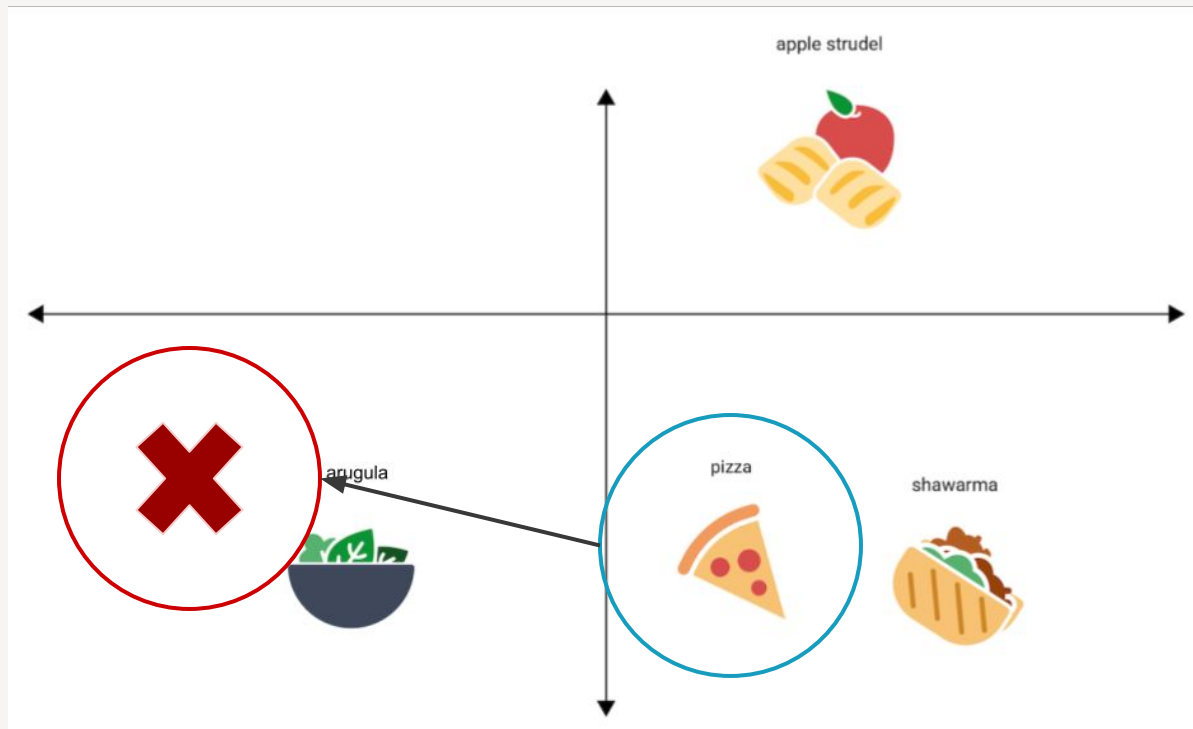
“This **pizza** is
totally
disgusting”



DYNAMIC EMBEDDING

“I ate **pizza**”

“This **pizza** is
totally
disgusting”



Skip-gram (word2vec) (static embedding)

- Idea básica: se entrena un modelo con pares **$[w, c]$** , que actualiza sus parámetros de acuerdo a si el par es real o no.
 - a. La palabra objetivo y sus vecinas en el contexto se consideran ejemplos positivos.
 - b. Se toman palabras random fuera del contexto para formar ejemplos negativos.
 - c. Se entrena un clasificador que distinga estos ejemplos.
 - d. Se usan los parámetros resultantes como embeddings para la palabra en cuestión.

Skip-gram (word2vec) (static embedding)

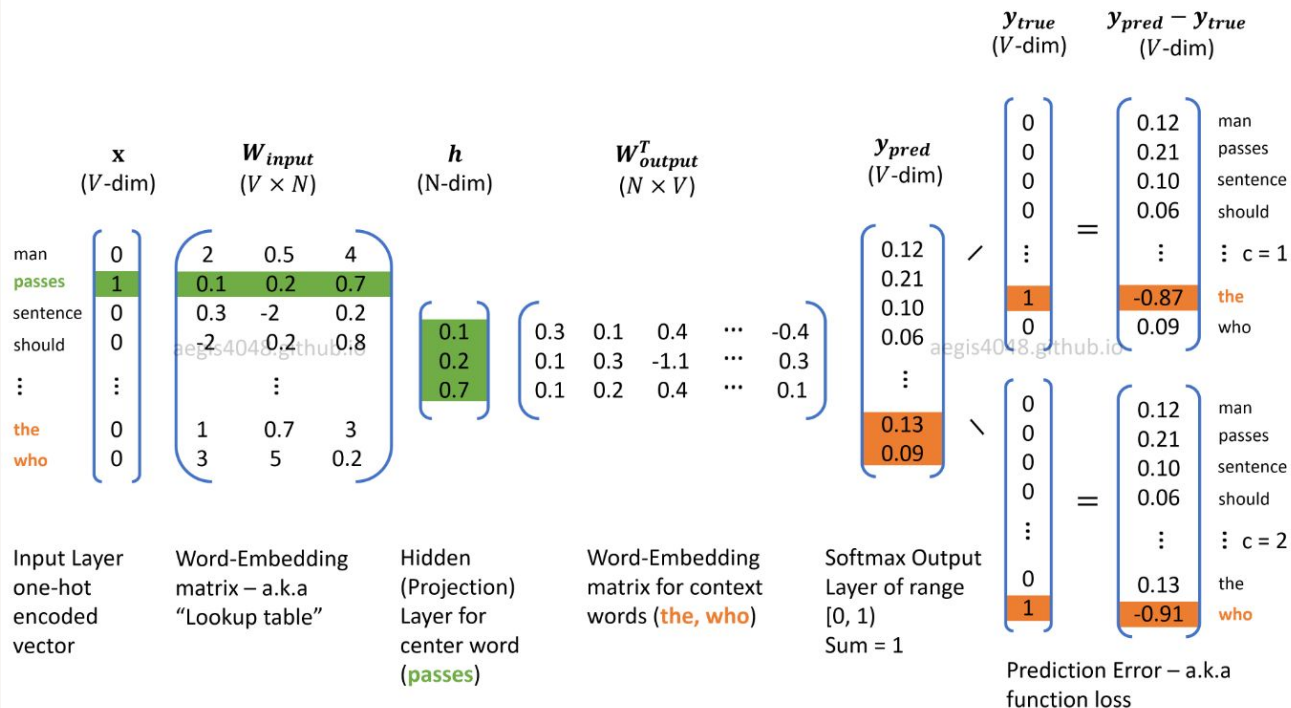
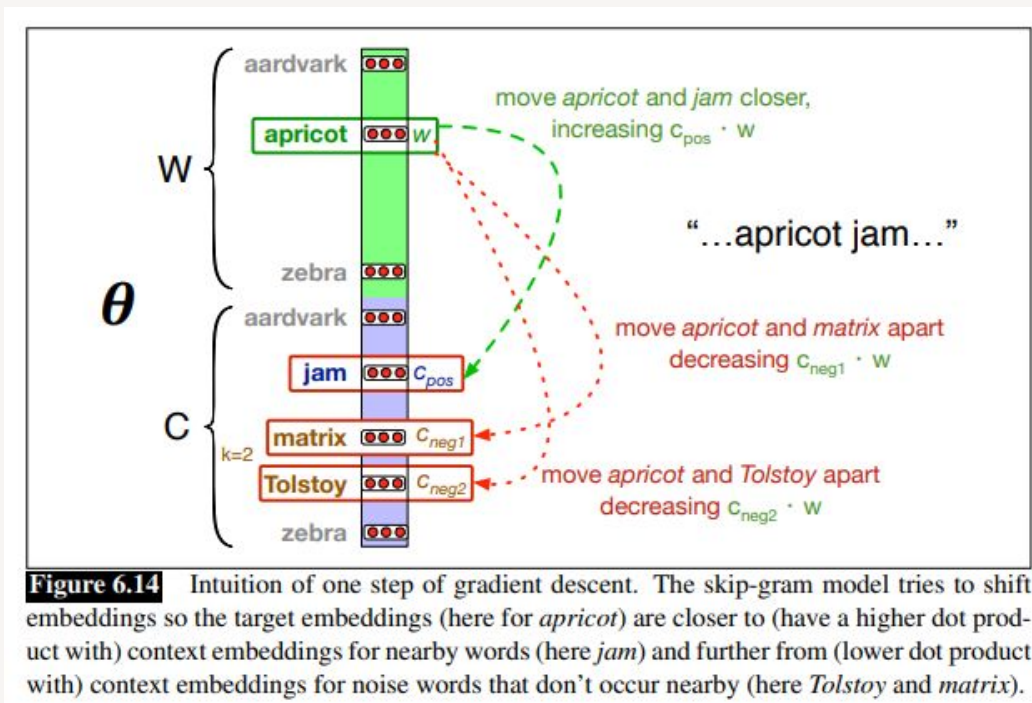
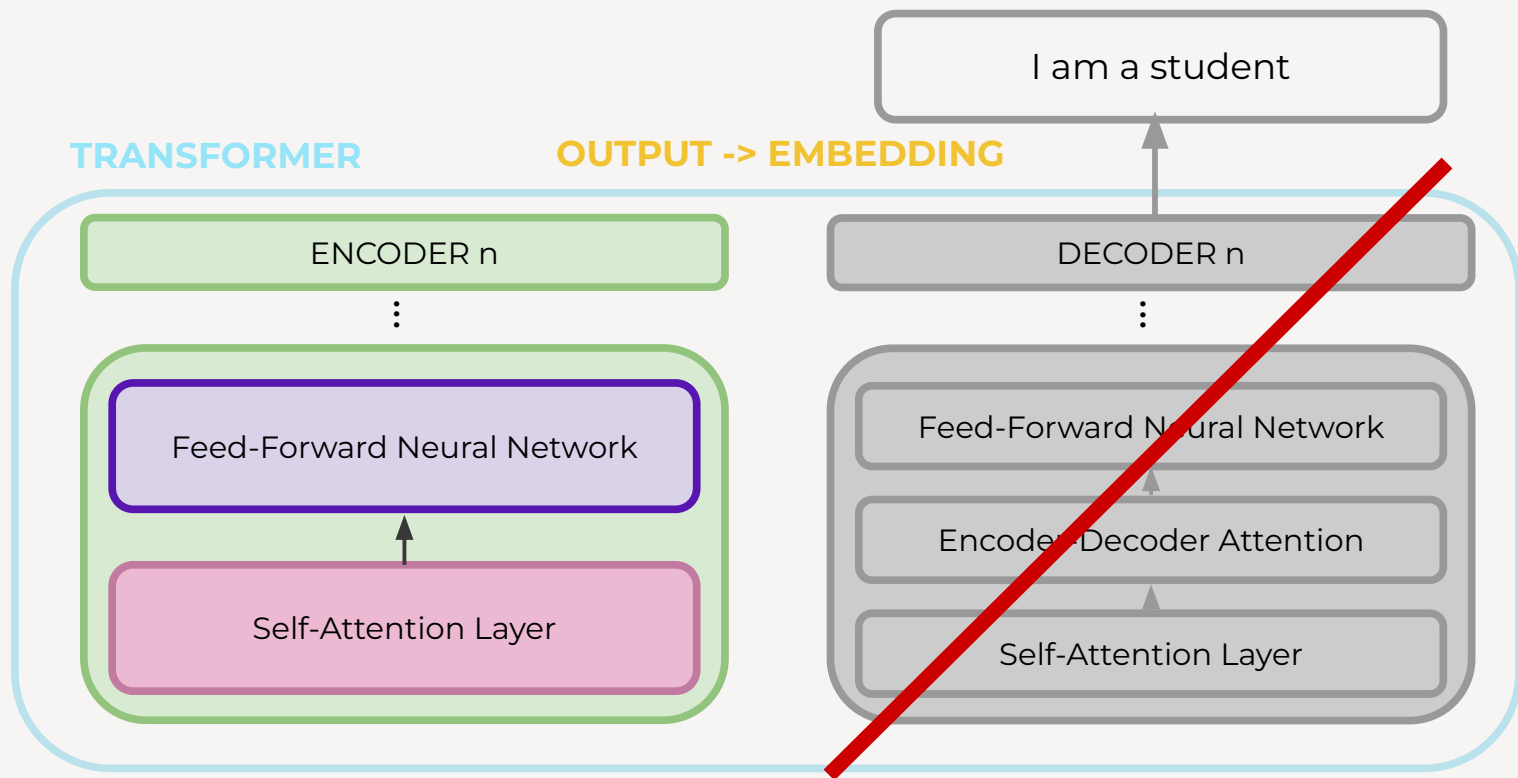


Figure 5: Skip-Gram model structure. Current center word is "passes"

Skip-gram (word2vec) (static embedding)



Encoder-only Transformer (dynamic embedding)



BERT (dynamic embedding)

Bidirectional Encoder Representations from Transformers

Procesa el texto basándose en el contexto previo y el futuro

Solo usa el stack de encoders (“Encoder-only Transformer”)

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

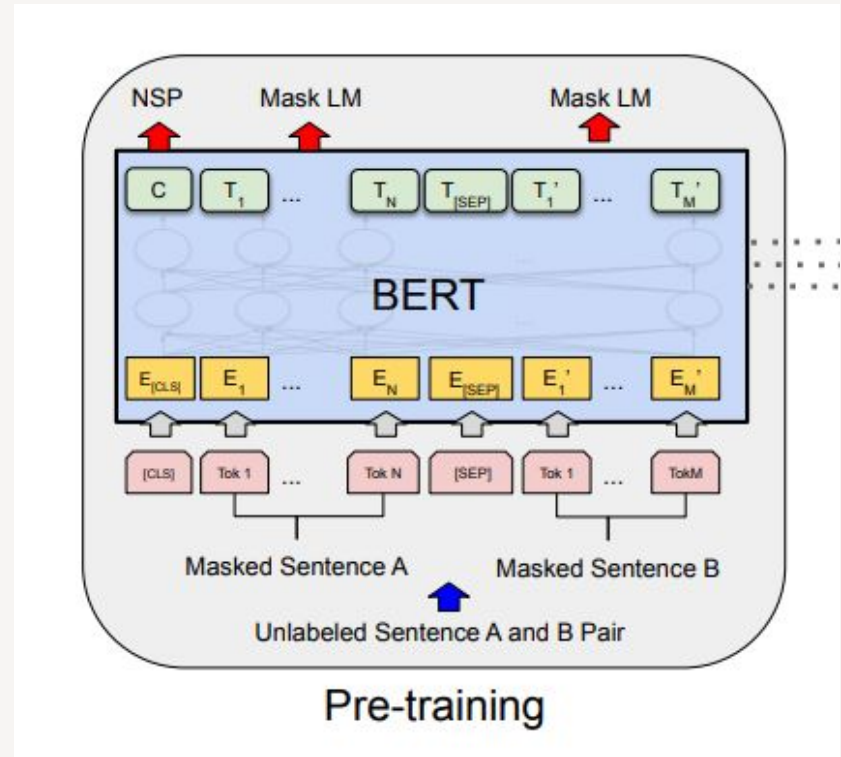
We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from

addition on both layers. As a result, the model can be fine-tuned on a wide range of tasks for a wide range of languages, with no need for task-specific architectures or output layer definitions. We demonstrate the substantial task-agnostic performance of BERT on a wide range of tasks.

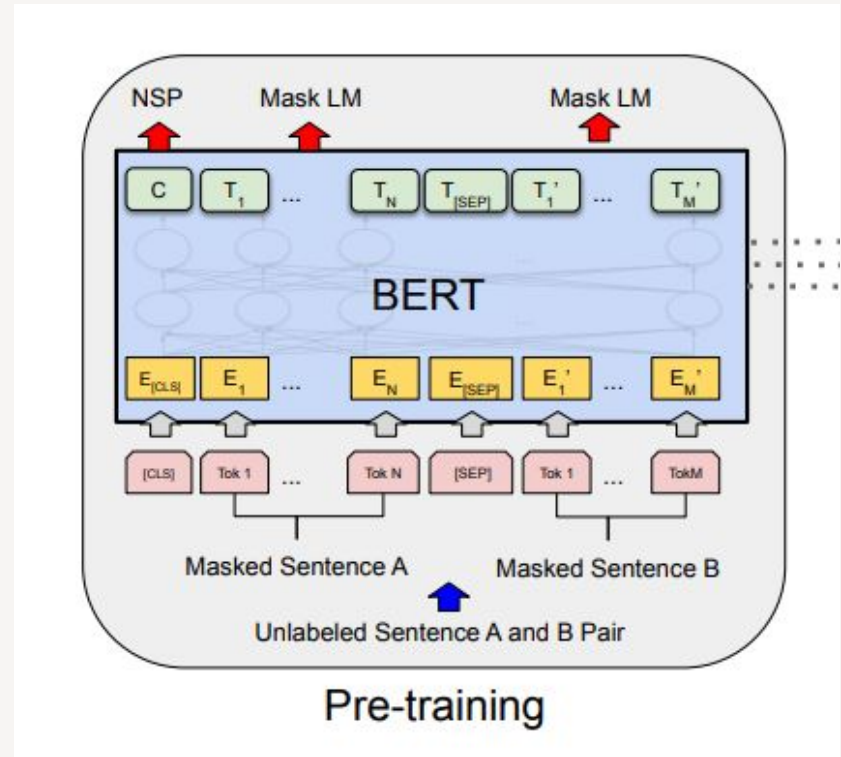
There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

Model Architecture BERT’s model architecture is a multi-layer bidirectional Transformer **encoder-based** on the original implementation described in Vaswani et al. (2017) and released in the `tensorflow` library.¹ Because the use

- **Atención:** sin enmascarar
- **Tarea de predicción:** NSP (*next sentence prediction*) y MLM (*masked language modeling*)
- **Input:** Dos fragmentos con separador



- **Atención:** sin enmascarar
- **Tarea de predicción:** NSP (*next sentence prediction*) y MLM (*masked language modeling*)
- **Input:** Dos fragmentos con separador



Tarea de predicción: MLM

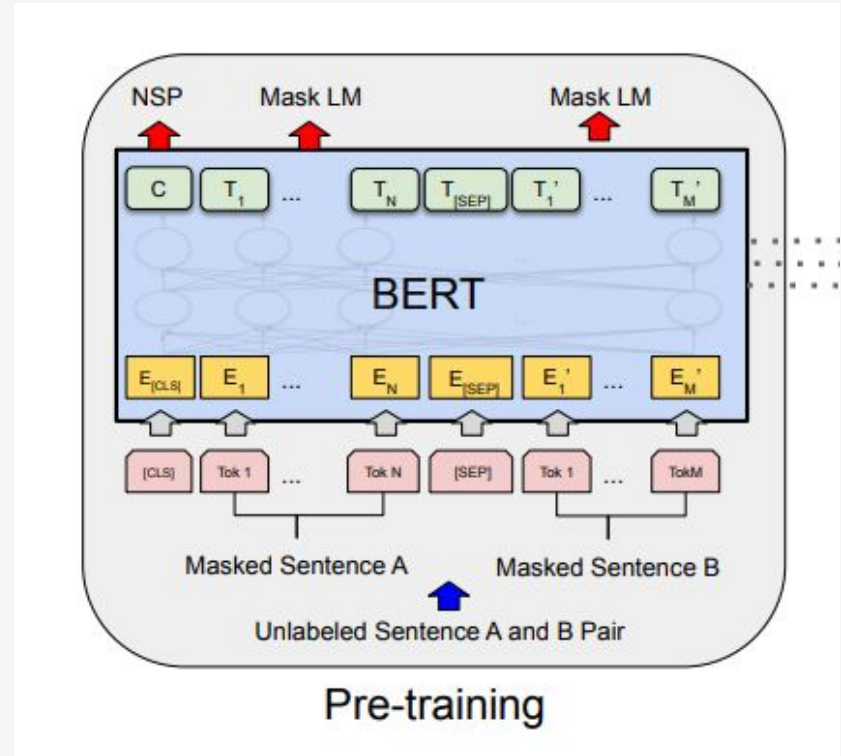
¿Cuántos enmascaro? 15%

¿Cómo? 80% [MASK], 10% random, 10% idénticos

“Pizza is a dish made with a flat, usually circular, dough made from wheat flour , yeast , water, and salt (sometimes olive oil) that is traditionally topped with tomato sauce and ...”

“Pizza is a [MASK] made with a flat, usually [CIRCULAR], dough made from wheat flour , yeast , water, and [MASK] (sometimes olive oil) that is [SUPERNOVA] topped with tomato sauce and ...”

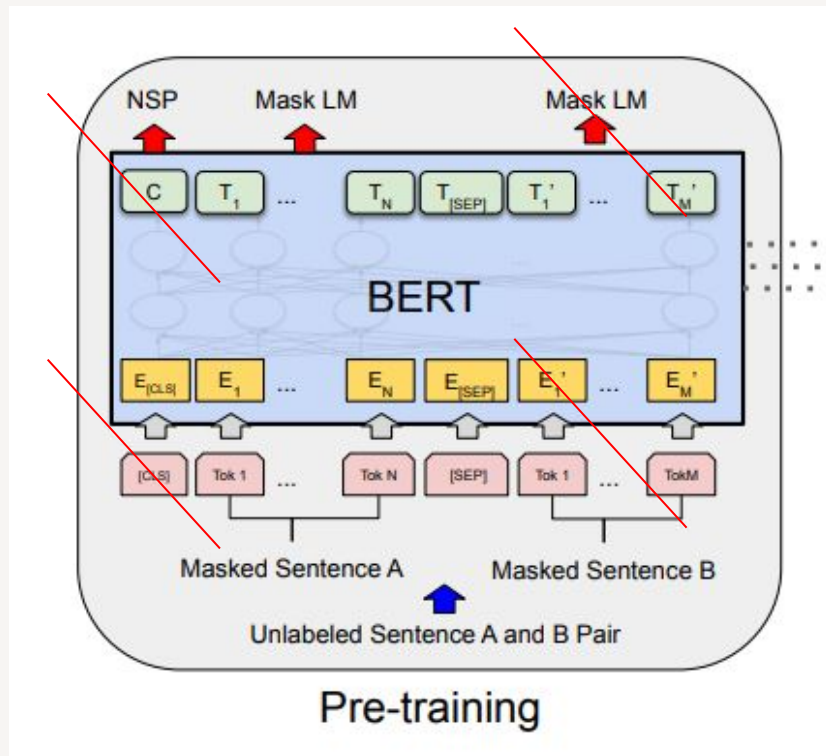
- **Atención:** sin enmascarar
- **Tarea de predicción:** NSP (*next sentence prediction*) y MLM (*masked language modeling*)
- **Input:** Dos fragmentos con separador



Simplificando BERT (dynamic embedding)

Tarea de predicción:

~~NSP (next sentence prediction)~~ y MLM (masked language modeling)



05

POSITIONAL ENCODING

¿Por qué ***positional encoding*** fue relevante?

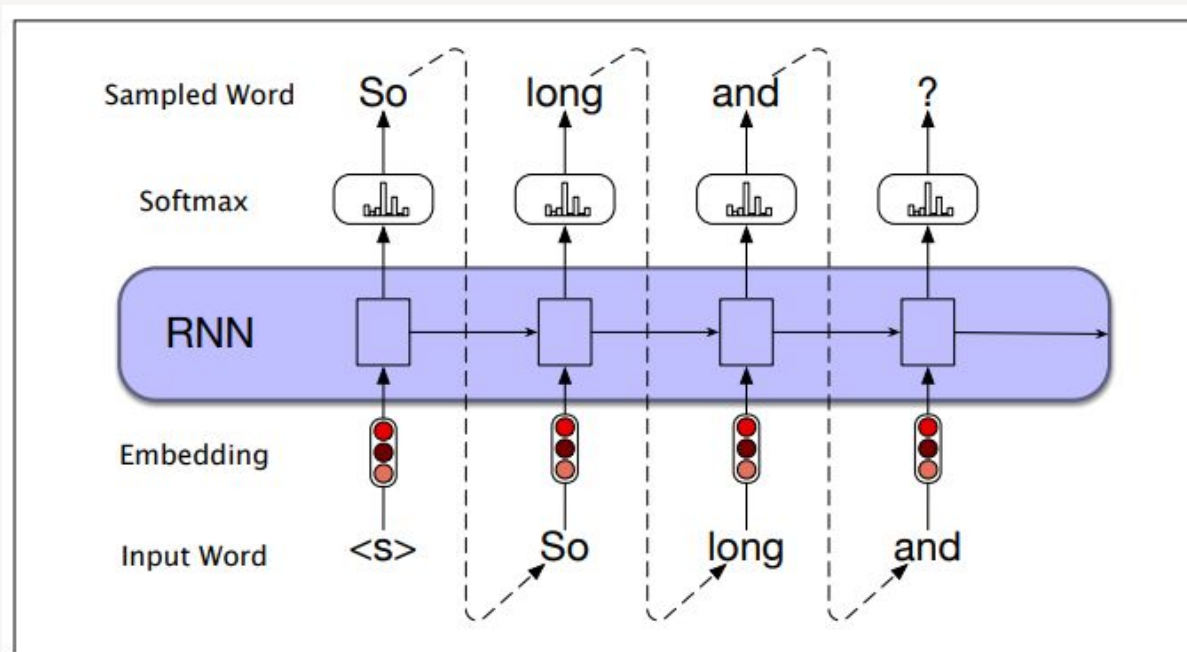


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

[In Speech and Language Processing \(3rd ed. draft\) \(Chapter 9\)](#)

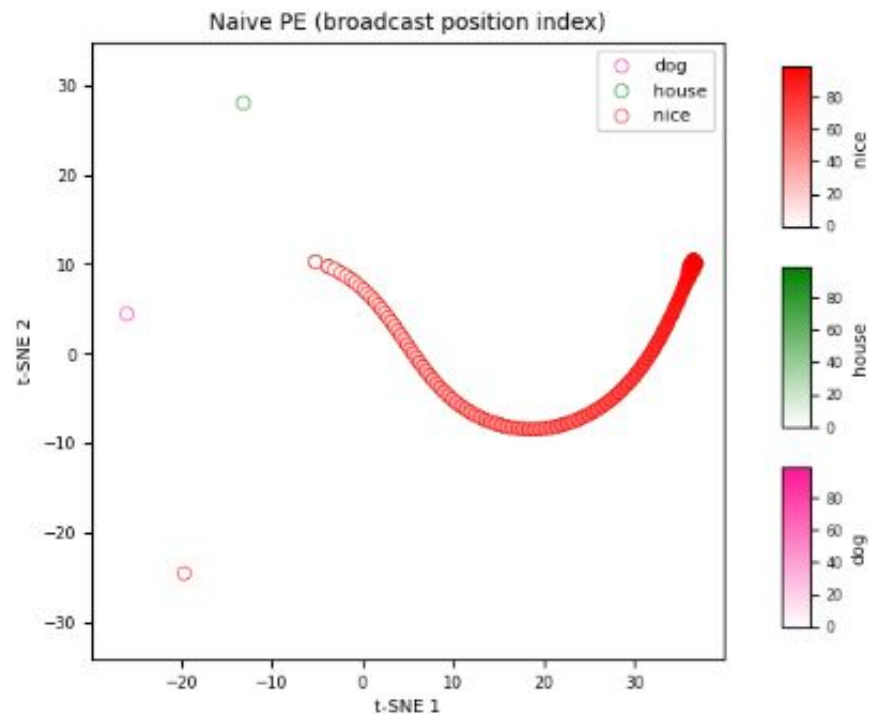
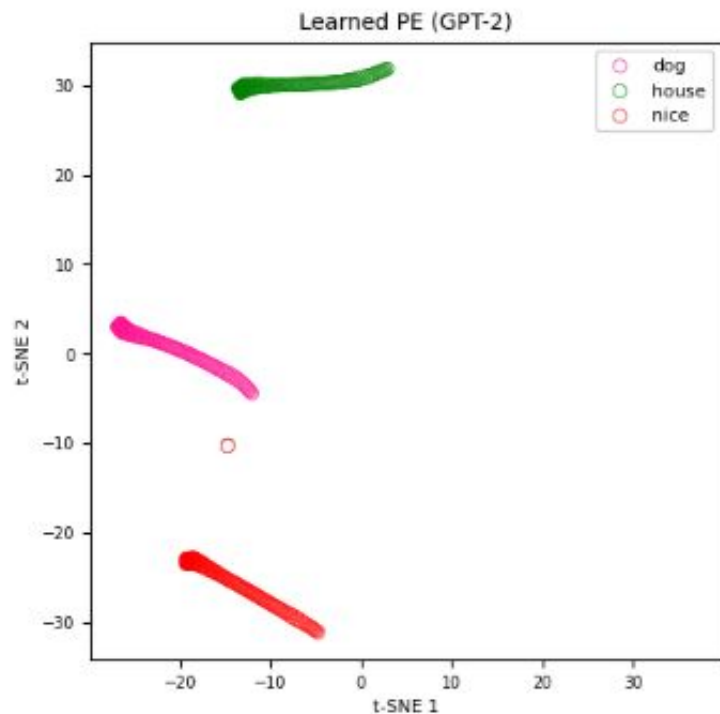
¿Por qué ***positional encoding*** fue relevante?

- Encoder/Decoder application for Machine Translation 
- Attention Mechanism for Machine Translation 
- Embeddings/Softmax/Loss Function/ ... 

¿Qué nos interesan al elegir una estrategia?

- ¿Cómo declaramos las posiciones?
- ¿Cómo impacta el esquema que usamos de *positional encoding* en la generalización del transformer?
 - Disclaimer: el transformer mismo puede imponer un límite pero acá nos interesa entender si dicho límite viene de la elección del *positional encoding*.

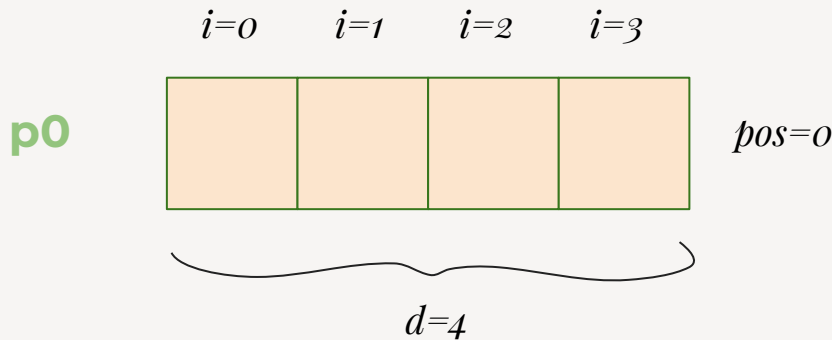
Three tokens at 100 positions, learned vs naive PE



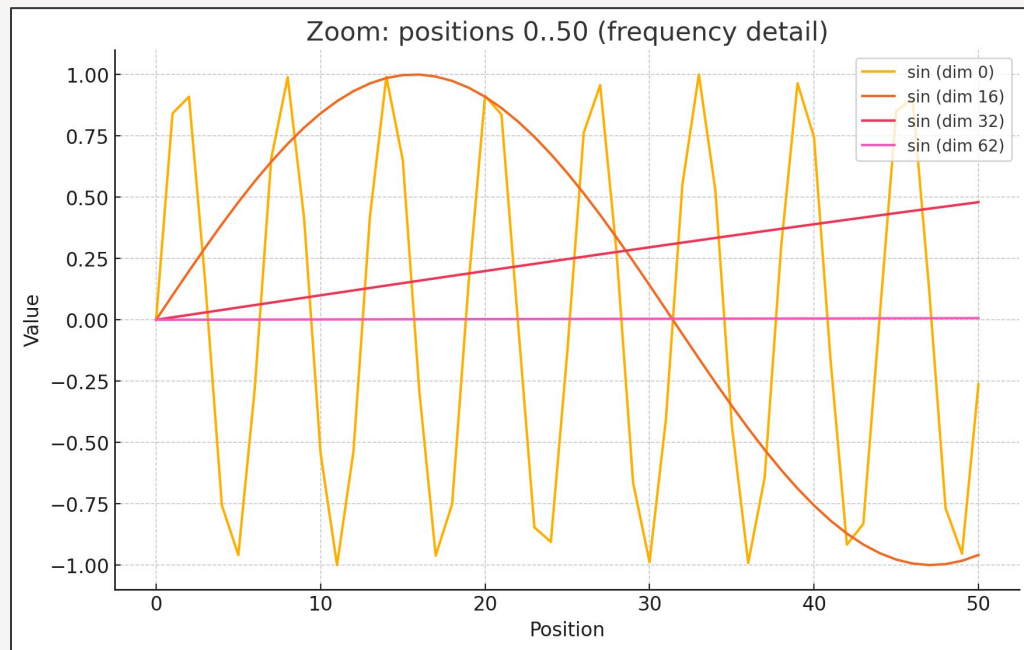
ABSOLUTE POSITIONAL ENCODING

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$



MOTIVACIÓN: RELACIONES A CORTA/LARGA DISTANCIA



Paper original: 512

[Kemal Erdem, \(May 2021\). "Understanding Positional Encoding in Transformers".
<https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers>](https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers)

¿Por qué es de interés procesar secuencias largas?

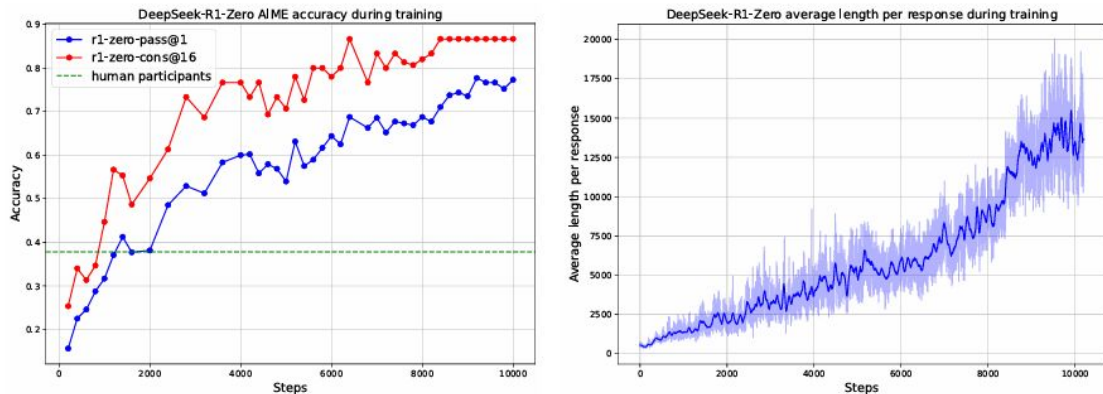


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

(clase 4!)

06

AVANCES IMPORTANTES

Mixture of Experts

Motivación: eficiencia para modelos a escala (separa capacidad del modelo de cómputo necesario para el input)

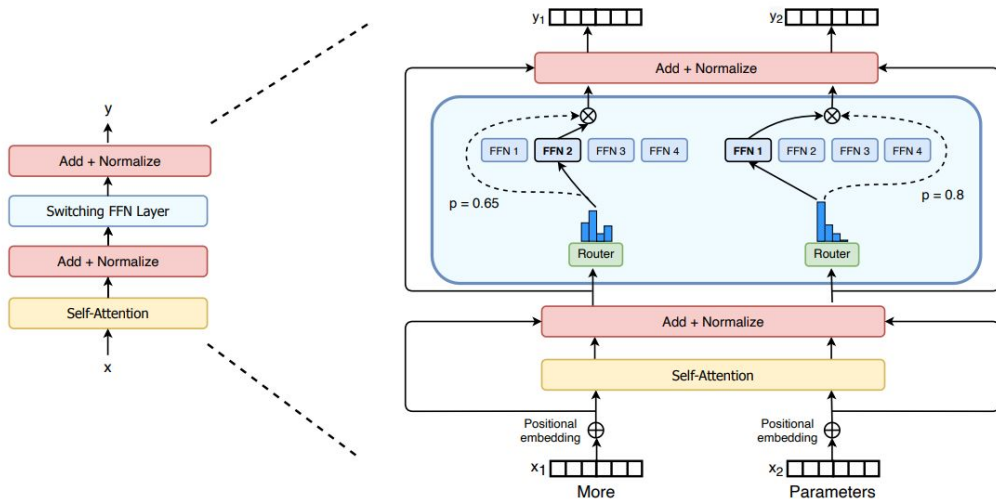


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = “More” and x_2 = “Parameters” below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Rotary Positional Encoding (y derivados)

Motivación: modelar posiciones de forma relativa para obtener mejor generalización



Figure 1: Needle in a Haystack evaluation. Green squares indicates a high retrieval success rate, the white dashed line denotes the longest length examples seen at training or finetuning, and the Y-axis represents the distance to the retrieved target.

[A Controlled Study on Long Context Extension and Generalization in LLMs](#)

06

BIBLIOGRAFÍA

BIBLIOGRAFÍA

Tokenizer

Open AI Platform Tokenizer <https://platform.openai.com/tokenizer>

TikToken Github <https://github.com/openai/tiktoken>

Google SentencePiece Github <https://github.com/google/sentencepiece>

Land S., Bartolo M. (2024). ***Fishing for Magikarp: Automatically detecting under-trained tokens in large language models*** arXiv. <https://arxiv.org/pdf/2405.05417>

Let's build the GPT Tokenizer <https://www.youtube.com/watch?v=zduSFxRajkE> (práctico) ★

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. & Sutskever, I. (2019). **Language models are unsupervised multitask learners.**

Summary of the tokenizers, https://huggingface.co/docs/transformers/tokenizer_summary

BIBLIOGRAFÍA

Embedding

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). ***Efficient estimation of word representations in vector space***. arXiv. <https://arxiv.org/abs/1301.3781>

RDevlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). ***BERT: Pre-training of deep bidirectional transformers for language understanding***. arXiv. <https://arxiv.org/pdf/1810.04805>

Jurafsky, D., & Martin, J. H. (2023). ***Speech and Language Processing*** (3rd ed. draft). Retrieved from <https://web.stanford.edu/~jurafsky/slp3/>

Walib Saba (2020) ***Word embeddings: The good, the bad, and the ugly***. Medium. Retrieved from <https://medium.com/ontologik/word-embeddings-the-good-the-bad-and-the-ugly-ad67ada0e688>

Wevi: word embedding visual inspector, <https://ronxin.github.io/wevi/>

BIBLIOGRAFÍA

Positional Encoding

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). **Attention is all you need**. arXiv. <https://arxiv.org/pdf/1706.03762> (RNN limitación)

Open AI Community **Positional Encoding Grammar**
<https://community.openai.com/t/positional-encoding-and-implicit-grammar/74564>

Jurafsky, D., & Martin, J. H. (2023). Part-of-Speech Tagging. **In Speech and Language Processing** (3rd ed. draft) (Chapter 9). Retrieved from <https://web.stanford.edu/~jurafsky/slp3/9.pdf>

Kazemnejad, A. (2019, September 20). **Transformer architecture: The positional encoding**.
https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

[How Rotary Position Embedding Supercharges Modern LLMs \[RoPE\]](#) ★

Federico Barbero, Alex Vitvitskyi, Christos Perivolaropoulos, Razvan Pascanu, Petar Veličković (2024) **Round and Round We Go! What makes Rotary Positional Encodings useful?**. arXiv. <https://arxiv.org/abs/2410.06205>

Recomendaciones para curiosos



Si todavía no te cierra del todo la explicación del transformer en general, mirate estos videos y después repasá las clases 1 y 2!

- Visualizing Attention, a Transformer's Heart, <https://www.3blue1brown.com/lessons/attention> ★
- How might LLMs store facts | DL7, <https://www.youtube.com/watch?v=9-Jl0dxWQs8> ★
(si querés irte al paper principal: <https://arxiv.org/abs/2402.07321>)

Consultar si te interesa un resumen de todo lo que vimos + algún que otro “extra” (adaptaciones modernas). El objetivo de ver esto sería entender dónde están los puntos flojos de entendimiento propio (casi todo debería ser familiar y sentir que se entiende, más allá de las dudas filosóficas)

- **How Transformer LLMs Work**,
<https://learn.deeplearning.ai/courses/how-transformer-llms-work/lesson/nfshb/introduction?courseName=how-transformer-llms-work>
(recomendado 1.5x o 2x, con excepción de las partes nuevas).

Recomendaciones para curiosos



Si te interesa mucho el área de Natural Language Processing (esta materia toca algunos conceptos pero está lejiiisimos de cubrir todo) consultar este libro.

- Jurafsky, D., & Martin, J. H. (2023). Part-of-Speech Tagging. ***In Speech and Language Processing*** (3rd ed. draft) <https://web.stanford.edu/~jurafsky/slp3/>
(MUY recomendado, las explicaciones son excelentes)

Si estás buscando algoritmos de embedding y no estás seguro de qué preguntas deberías hacerte, este blog post tiene una buena guía (ver las explicaciones al final) y [HuggingFace](#) es excelente fuente para proyectos

- Best Embedding Models of 2025, <https://slashdot.org/software/embedding-models/>
- Sentence Transformers <https://www.sbert.net>
- Quickstart - Sentence Transformers <https://www.sbert.net/docs/quickstart.html#sentence-transformer>

Recomendaciones para curiosos



Si te interesa profundizar en la historia del salto RNNs a Transformer (y ver de dónde salen todas las ideas a partir de transformers, mucho está reutilizado!) ver en orden:

- RNNs and LSTMs (Capítulo 8), <https://web.stanford.edu/~jurafsky/slp3/8.pdf>
- The Transformer (Capítulo 9), <https://web.stanford.edu/~jurafsky/slp3/9.pdf>

Si te interesa meterte en riesgos de seguridad asociados a tokenización

- **SolidGoldMagikarp (plus, prompt generation)**, <https://www.lesswrong.com/posts/aPeJE8bSo6rAFoLqg/solidgoldmagikarp-plus-prompt-generation>
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, Matt Fredrikson, **Universal and Transferable Adversarial Attacks on Aligned Language Models** (2023), arXiv, <https://arxiv.org/abs/2307.15043>

Recomendaciones para curiosos

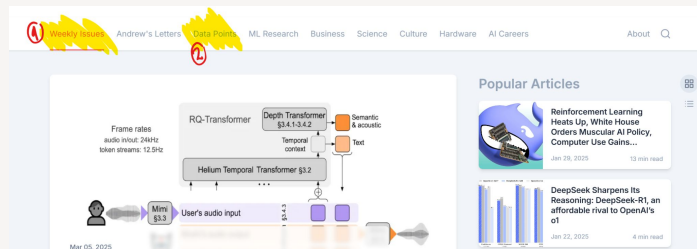


Técnica más utilizada para positional encoding hoy

- [RoPE \(Rotary positional embeddings\) explained: The positional workhorse of modern LLMs](#)
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, Yunfeng Liu (2021) **RoFormer: Enhanced Transformer with Rotary Position Embedding**, arXiv, <https://arxiv.org/abs/2104.09864>

Si te interesa estar MÁS O MENOS al día con noticias que mezclan tanto industria como investigación, con un foco más pesado en industria de modelos fundacionales, recomendado suscribirse a:

- <https://www.deeplearning.ai/the-batch/>



**Video/Paper/Post
sugerido**

¿Cómo podrían los LLM
almacenar hechos?



Pregunta abierta

¿Por qué los modelos de
lenguaje son buenos
siguiendo
instrucciones?

¿Qué vemos la
clase próxima?